Lesson plan · for the teacher

Exams without clashes

Optimization · Level 3 · Lesson 2

Time
40 min

Materials
Chalk, or string and eight chairs; Coloured paper or caps for the slots; A list of exams and who takes each

Learning goals

- Draw exam clashes as a map and colour it with few slots.
- Prove the fewest slots with a clique, or a solver's OPTIMAL.
- Weigh a fairness rule against a short exam week.

Activity: The chalk clash map

1. Warm-up (5 min): Ask whether anyone ever had two exams at the same time or three in one morning. List what made exam weeks hard.
2. Teach (8 min): Write six to eight exams on the board with who takes each. Draw a dot per exam and a line between two exams that share a pupil. Explain: same colour = same slot, so joined dots need different colours.
3. Draw and colour (15 min): Groups copy the map (on paper or in chalk outside) and colour it with as few colours as they can. Each group writes its number of slots on the board.
4. Prove it (7 min): Ask: can anyone do it with fewer? Find a group of exams that all clash with each other. Its size is the fewest possible, whatever anyone tries.
5. Wrap-up (5 min): Add the rule "no pupil sits two slots in a row" and recount. Ask which plan the class prefers, the shorter week or the rested pupils, and why.

Use the lab's eight exams if you have no real list: maths, English, science, Arabic, history, geography, art and French, with the six pupils in the "Exam slots without clashes" lab.

Discussion

- How do we know nobody can use fewer slots?
- Who had the hardest exam days in our plan?
- Is a shorter exam week worth three tired pupils?

Exit ticket

- Draw a clash map of four exams that needs exactly three slots, and say why.

Try it in class tomorrow
List next term's exams and which classes or pupils take each. Let pupils draw the clash map with chalk dots and lines (or string between chairs), then colour it with as few slots as they can. Ask: who has two exams in a row, and is that fair?

Worksheet · for pupils

Exams without clashes

Optimization · Level 3 · Lesson 2

Name _____ Class _____ Date _____

1 **The exam model says 3 slots, OPTIMAL. But Tarek (science, Arabic, French) now sits three exams back to back, and his parents complain. What should the school do?**

- ◯ Nothing: OPTIMAL can't be improved
- ◯ Agree a fairness rule, add it, and solve again
- ◯ Leave Tarek out of the model

2 **Hala takes history, geography and art. Dina takes Arabic and art. Which pairs of exams must be in different slots? Choose all that apply.**

- ☐ History and art
- ☐ Geography and art
- ☐ Arabic and art
- ☐ History and Arabic
- ☐ Geography and Arabic

3 **Greedy colouring used 4 slots, so at least 4 slots are needed.**

- ◯ True
- ◯ False

4 **Four exams, numbered 0 to 3 (maths, science, Arabic, art), and three pupils who take exams [0, 1, 2], [1, 3] and [2, 3]. The model already makes the last slot as early as possible. Add the rule that each pupil's exams are all in different slots, then run it: how few slots are needed?**

```
from ortools.sat.python import cp_model
m = cp_model.CpModel()
slot = [m.new_int_var(1, 4, "") for e in range(4)]
last = m.new_int_var(1, 4, "last")
# each pupil's exams in different slots:
# [0, 1, 2], [1, 3] and [2, 3]
for s in slot:
    m.add(s <= last)
m.minimize(last)
solver = cp_model.CpSolver()
solver.solve(m)
print("slots:", solver.value(last))
```

Write your program:

5 **This exam model prints “slots used: 1”, but the exams clearly need more. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
m = cp_model.CpModel()
slot = {e: m.new_int_var(1, 8, e) for e in exams}
last = m.new_int_var(1, 8, "last")
for taken in pupils:
    m.add_all_different([slot[e] for e in taken])
for e in exams:
    m.add(slot[e] >= last)
m.minimize(last)
```

✎ **In your school’s last exam week, who had the hardest timetable? Would a written fairness rule have helped, and what would it have cost?**